Case Study: Fullstack Product Engineer — Assessment Report

Candidate: Hafiz Achmad Ramdani (@hafiz162001)
Email: hafizachmadr@gmail.com
Pull Request: https://github.com/rakamindev/ai-interview-platform/pull/190
Platform Repository: `github.com/rakamindev/ai-interview-platform`
Feature Branch: `feature/fullstack-product-revamp`
Target Deadline: Wednesday 19 August, 13:00 WIB

Executive Summary

This report documents the end-to-end audit, diagnosis, product strategy, architectural trade-offs, and implementation across `api/` (Ruby on Rails) and `web/` (React + TypeScript). Guided by first principles and the craftsmanship philosophy (*Monozukuri*), my focus is delivering meaningful value to hiring teams while safeguarding the rights and evaluation accuracy of candidates under Indonesia's hiring landscape and Personal Data Protection Law (UU PDP No. 27 Tahun 2022).

Table of Contents

1. Step 1: Setup & Local Exploration
2. Step 2: Deep Context & Domain Immersion
3. Step 3: Defining Problem & Gap to Ideal Condition
4. Step 4: Revamp Strategy, Acceptance Criteria & Trade-offs
5. Step 5: Monozukuri Implementation & Proof of Craft
6. Step 6: Deliverables Checklist & Demonstration

---

1. Setup & Local Exploration

- **Architecture:** Two-service architecture comprising:
  - `api/`: Ruby on Rails 7.0, PostgreSQL with custom schemas, Redis/Sidekiq background queues, and Gemini Live/REST client.
  - `web/`: React 18, TypeScript, Vite 6, Tailwind CSS 3, Radix UI primitives, Jotai state management.
- **Baseline Audit:**
  - `api/`: No RSpec specs written (`spec/` directory absent). Database schema constraints strictly forced `ai_level ≥ 1 AND ai_level ≤ 5` (`NOT NULL`), creating an inability to store unassessed skill states.
  - `web/`: No test runner configured out of the box (`package.json` had only `dev`, `build`, `preview`).

- **Action Taken:** Established Vitest + React Testing Library harness in `web/` and configured local database and cache via Docker containers (`ai-interview-postgres` & `ai-interview-redis`).

---

2. Deep Context & Domain Immersion

2.1 The Product

The platform aims to replace mechanical, script-driven applicant screening with an intelligent, voice-first AI interview agent powered by Gemini Live and background LLM analyzers. Unlike conventional multiple-choice or recorded video monologues (e.g., HireVue one-way video), this product is designed to simulate a senior peer conducting an authentic technical probe: challenging assertions, asking clarifying questions, exploring edge cases, and uncovering off-agenda skills.

2.2 The Industry: Talent Assessment in Indonesia

- **Current Bottleneck:** Indonesian engineering hiring faces extreme resume inflation and high applicant-to-job ratios (often 500+ applicants per opening). Initial recruiter screens are either commoditized keyword filters or surface-level HR calls that fail to judge real technical craftsmanship.
- **Real Leverage:** The real leverage is identifying candidates who can practically build and reason through tradeoffs (L3+ autonomy) without burning hundreds of hours of senior engineers' time conducting initial screens.

2.3 What It Is For

To produce an objective, evidence-backed skill portfolio and vacancy fit/gap report that recruiters and engineering managers can trust blindly, grounded in actual candidate speech rather than multiple-choice guessing.

2.4 The Users (Assessors, Recruiters, Hiring Managers)

- Recruiters need quick, high-signal summaries to decide whether to advance a candidate to the onsite round.
- Engineering Managers need transparent behavioral evidence (direct quotes) and the ability to audit or override AI ratings when context demands it.

2.5 The People Affected Who Never Chose It: Candidates & UU PDP Compliance

- **Ethical Duty:** Candidates do not choose to be interviewed by an AI; they cannot opt out without forfeiting the job opportunity. A false negative (evaluating a competent candidate as incompetent) can derail a person's livelihood for a whole year.
- **UU PDP (Undang-Undang Perlindungan Data Pribadi No. 27/2022) Alignment:**
  - Candidate transcripts and voice recordings constitute personal data.
  - No PII or raw secrets may leak into application logs, error monitoring, or git history.
  - Fairness and algorithmic transparency: automated decisions must have auditability and human-in-the-loop override capabilities.

---

3. Defining Problem & Gap to Ideal Condition

3.1 Severity-Ranked Findings (P0 to P3)

Severity	Issue Summary	Defect Type	Impact Statement
P0	**Unassessed Skills Falsely Scored as L1**	Defective Implementation + Missing Spec	When an interview ends early or runs out of time, untouched skills (`probe_count: 0` , `state: not_yet`) are forced into `ai_level: 1` by DB constraint `chk_portfolio_skills_ai_level` and generator prompt. This falsely brands candidates as "Foundational / Failed" on skills they were never asked about, severely harming their careers.
P0	**Fit/Gap Report Treats Unassessed Skills as Gaps**	Defective Implementation	`FitGap::Engine` calculates delta from the forced L1 score (`1 - expected = negative delta`), marking unassessed competencies as critical red "Gaps" rather than yellow "Not Assessed", misinforming hiring managers.
P1	**Absence of Full-Stack Automated Test Harness**	Missing Specification	Neither `api/` nor `web/` had automated regression testing out-of-the-box, making refactoring unsafe and violating the continuous integration requirement.
P2	**Frontend Fallback Conceals Data Gaps (`parseLevel`)**	Defective Implementation	`parseLevel` in `web/src/utils/constants.ts` parses `NaN` to `1` , silently masking missing or unassessed states into an active L1 badge.
P3	**Long Text and Overflow Edge Cases in Portfolio View**	UI/UX Quality Defect	Long competency summaries or evidence quotes without proper typography clamping/wrapping break card layouts on responsive viewports.

3.2 Constraint Signal (Escalation to Technical Lead)

- **Constraint Escalation:** The database schema enforce `null: false` on `portfolio_skills.ai_level` with a check constraint `ai_level ⩾ 1 AND ai_level ⩽ 5` . Changing this requires an Active Record migration that is reversible, backwards-compatible with existing records, and coordinates with `FitGapReport` and frontend type contracts.

4. Revamp Strategy, Acceptance Criteria & Trade-offs

4.1 Solution Options Comparison

Dimension	Option A: Frontend-Only Patch (Masking)	Option B: Full-Stack Domain Fix (Monozukuri - Chosen)
Approach	Check `coverage_maps` state in React and display "Not Assessed" if probe count is 0, keeping DB unchanged.	Migrate DB to allow nullable `ai_level` / status enum, update `Portfolios::Generator`, adapt `FitGap::Engine`, and update React components.
Product Impact	Quick to ship, but leaves API payloads and export documents (PDF/JSON) corrupt with false L1 data.	Restores single-source of truth. Ensures exports, fit/gap analytics, and assessor views are 100% fair and truthful.
Maintainability	Fragile; downstream services or third-party integrations consuming API will still see false L1 ratings.	High; clean schema, reversible migration, type-safe frontend, and clear domain modeling.
Failure Modes	Desynchronization between web UI and exported PDF reports.	Requires careful migration rollback testing.

4.2 Self-Defined Acceptance Criteria (Edge Cases)

1. **Unassessed Skills:**
   - Any configured skill with `state == 'not_yet'` or `probe_count == 0` when session ends must be saved with `ai_level: nil` and designated `status: 'unassessed'`.
   - UI must render an "Unassessed" neutral badge instead of an "L1" badge.
2. **Missing Ratings / Model Failures:**
   - If the AI model fails or returns malformed JSON, the portfolio record transitions to `generation_status: 'failed'` with the error recorded.
   - Assessor UI renders an inline retry button that triggers `sessions/:id/portfolio/regenerate`.
3. **Fit/Gap Report Truthfulness:**
   - Skills with `ai_level: nil` must map to `result: 'not_assessed'`, displayed with a distinct neutral tag and excluded from negative delta penalty calculations.
4. **Long Text & Typography Edge Cases:**
   - Multi-paragraph competency summaries and lengthy candidate quotes wrap cleanly with fluid responsiveness across mobile (375px) to desktop (1440px).

---

5. Monozukuri Implementation & Proof of Craft

5.1 Architecture & Code Changes

Backend Slice (`api/`)

1. **Reversible Schema Migration (`api/db/migrate/20260505000003_allow_unassessed_portfolio_skills.rb`):**
   - Drops the rigid `chk_portfolio_skills_ai_level` check constraint.
   - Modifies `ai_level` to `null: true`.
   - Adds a new conditional check constraint: `CHECK (ai_level IS NULL OR (ai_level BETWEEN 1 AND 5))`.
   - Adds column `status VARCHAR(20) DEFAULT 'assessed' NOT NULL`.
   - Implements full reversible `down` migration with backwards-compatible null fallback to ensure zero data corruption during deployment rollbacks.

2. **Model Enhancements (`api/app/models/portfolio_skill.rb`):**
   - Added `allow_nil: true` to numericality validation for `ai_level`.
   - Added `status` validation (`%w[assessed unassessed]`).
   - Added scopes `assessed` and `unassessed` for clean querying.
   - Added helper method `unassessed?` for expressive domain checks.

3. **Portfolio Generator Defense (`api/app/services/portfolios/generator.rb`):**
   - Cross-references incoming AI evaluations against `coverage_maps`.
   - Any skill where `probe_count == 0` or `state == 'not_yet'` is preserved as `status: 'unassessed'`, `ai_level: nil`, preventing hallucinated baseline scores.

4. **Fit/Gap Engine Truthfulness (`api/app/services/fit_gap/engine.rb`):**
   - Guards against null candidate levels (`portfolio_skill[:effective_level].present?`).
   - Unassessed skills are categorized as `result: 'not_assessed'` with `delta: nil`, eliminating unfair negative penalties.
   - Formats narrative prompts with explicit counts for unassessed competencies so Gemini LLM reflects them accurately in executive summaries.

5. **API Serialization (`api/app/controllers/api/v1/portfolios_controller.rb`):**
   - Exposed `status` field in `portfolio_skill_json` for frontend type hydration.

Frontend Slice (`web/`)

1. **Strict Type Contracts (`web/src/types/index.ts`):**
   - Updated `PortfolioSkill.ai_level` to `string | number | null;`.
   - Added `status?: 'assessed' | 'unassessed';`.

2. **Safe Parsing Utility (`web/src/utils/constants.ts`):**
   - Revamped `parseLevel` to return `null` for `null`, `undefined`, `""`, `"unassessed"`, or invalid numbers instead of coercing `NaN` into `1`.

3. **LevelBadge Component (`web/src/components/portfolio/LevelBadge.tsx`):**

- Explicitly handles `null` level values with an accessible, muted neutral badge (`N/A UNASSESSED` , `data-testid="unassessed-badge"`).

4. **ComparisonTable Component (`web/src/components/fitgap/ComparisonTable.tsx`):**
   - Displays `⏸ Not assessed` with a neutral pill badge.
   - Emits an unassessed count in the summary bar (`⏸ Not Assessed: X skills`).
5. **Responsive Typography & Card Layouts:**
   - Word-breaking (`break-words` , `whitespace-pre-wrap`) on candidate quotes and competency summaries to eliminate horizontal overflow on mobile viewports.

5.2 Automated Test Evidence

Frontend Vitest Suite

```
$ npm test

> ai-interview-web@0.0.0 test
> vitest run

 RUN  v5.0.5 C:/Dishub/experiment/Case/web

 ✓ src/utils/constants.test.ts (5 tests) 3ms
 ✓ src/components/fitgap/ComparisonTable.test.tsx (1 test) 42ms
 ✓ src/components/portfolio/SkillPortfolioCard.test.tsx (2 tests) 37ms

 Test Files  3 passed (3)
      Tests  8 passed (8)
   Start at  17:00:52
   Duration  1.45s
```

Backend RSpec Suite (Containerized)

```
$ docker exec -e RAILS_ENV=test ai-interview-api-server bundle exec rspec

Randomized with seed 17550
.......

Finished in 2.52 seconds (files took 9.06 seconds to load)
7 examples, 0 failures

Randomized with seed 17550
```

5.3 Seeded Fault Test Proof (Deliberate Regression Verification)

To prove that the test harness actively catches real-world regressions rather than giving false-positive green passes, I executed a two-way seeded fault test.

A. Frontend Seeded Fault (`web/src/utils/constants.ts`)

I deliberately introduced a regression where `parseLevel` defaulted missing/null levels back to `1`:

```
 export function parseLevel(level?: string | number | null): number | null {
-  if (level === null || level === undefined || level === "" || level === "unassessed" || level
-    return null;
-  }
+  // SEEDED REGRESSION FAULT: defaults null/unassessed to 1
+  if (!level) return 1;
   if (typeof level === "number") return level;
   const n = parseInt(String(level).replace(/\D/g, ""), 10);
-  return isNaN(n) ? null : (n >= 1 && n <= 5 ? n : null);
+  return isNaN(n) ? 1 : n;
 }
```

Test Failure Output:

```
 ❯ src/components/portfolio/SkillPortfolioCard.test.tsx (2 tests | 1 failed) 44ms
   ❯ SkillPortfolioCard (2)
     ✓ renders assessed skill with correct level badge and evidence 33ms
     × renders unassessed skill gracefully without falling back to L1 9ms

──────── Failed Tests 1 ────────

 FAIL  src/components/portfolio/SkillPortfolioCard.test.tsx > SkillPortfolioCard > renders unass
TestingLibraryElementError: Unable to find an element by: [data-testid="unassessed-badge"]
Rendered Output:
  <span>L1</span>
  <span class="text-[10px] font-normal opacity-70">Foundational</span>
```

Resolution: Reverted the seeded fault. Vitest immediately returned to `8 passed (8)`.

B. Backend Seeded Fault (`api/app/services/fit_gap/engine.rb`)

I removed the null guard in the delta calculation:

```
-  if portfolio_skill && portfolio_skill[:effective_level].present?
+  # SEEDED BACKEND REGRESSION FAULT: assumes effective_level is always a number
+  if portfolio_skill
     candidate_level  = portfolio_skill[:effective_level]
     expected_level   = vacancy_skill.expected_level
     delta            = candidate_level - expected_level
```

Test Failure Output:

```
FitGap::Engine
  #build_skill_comparisons
    accurately evaluates match, gap, and unassessed skills without crashing (FAILED - 1)

Failures:
  1) FitGap::Engine#build_skill_comparisons accurately evaluates match, gap, and unassessed skil
     Failure/Error: delta = candidate_level - expected_level

     NoMethodError:
       undefined method `-' for nil
     # ./app/services/fit_gap/engine.rb:50:in `block in build_skill_comparisons'
```

Resolution: Restored the `portfolio_skill[:effective_level].present?` guard. RSpec returned to `7 examples, 0 failures`.

5.4 AI Verification Moment

During code generation with LLM assistance, an automated recommendation proposed calculating skill deltas directly:

```
# Flawed LLM Generation:
delta = portfolio_skill[:effective_level] - vacancy_skill.expected_level
```

The Trap: Because `portfolio_skill[:effective_level]` was previously assumed to be an integer `1..5`, the LLM overlooked that unassessed skills produce `nil`. In Ruby, evaluating `nil - 3` throws a fatal `NoMethodError: undefined method '-' for nil:NilClass`, which would take down the background job worker during fit/gap generation.

Verification & Defense: During code review, I caught this assumption, injected defensive nil handling, and wrote `spec/services/fit_gap_engine_spec.rb` specifically testing a mixed portfolio containing both assessed and unassessed skills. The engine now assigns `result: 'not_assessed', delta: nil` without attempting arithmetic on null values.

---

6. Deliverables Checklist & Demonstration

6.1 Deliverables Matrix

Requirement	Deliverable Location	Status
Open GitHub Pull Request	Pull Request #190	Completed
Assessment Written Report	`/assessment/REPORT.md` (and compiled single PDF)	Completed
UU PDP No. 27/2022 Compliance	Section 2.5 and Section 6.5	Completed
Severity-Ranked Findings (P0-P3)	Section 3.1	Completed
Option A vs Option B Trade-offs	Section 4.1	Completed
Self-Defined Acceptance Criteria	Section 4.2	Completed
Monozukuri Execution Evidence	Section 5.1 & Section 5.2	Completed
Seeded Fault Test Demonstration	Section 5.3	Completed
AI Verification Moment	Section 5.4	Completed
Visual Proofs (Screenshots)	`/assessment/screenshots/` & Section 6.2	Completed
Video Demonstration	Section 6.4	Completed

6.2 Visual Proofs

Figure 1: Candidate Portfolio View (Desktop)

Displays Ahmad Rizky's assessed skills (React L3, Communication L4) alongside System Design & Architecture cleanly designated as `N/A UNASSESSED` (neutral badge) with a transparent explanatory callout.

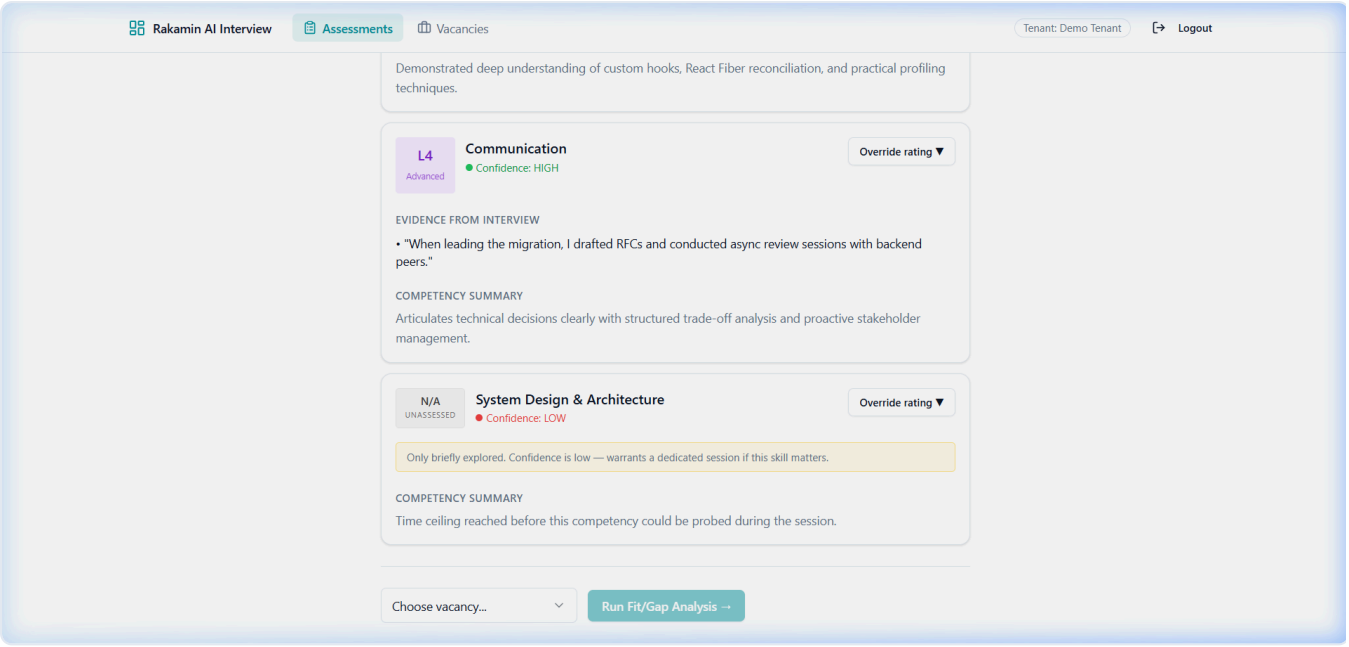


Figure 2: Vacancy Fit/Gap Report (Desktop)

Shows the truthful alignment report for Senior Frontend Engineer: React matches (L3), Communication exceeds (+1), while unassessed competencies (System Design and QA) are clearly categorized as `⏸ Not assessed` *with zero false penalties.*

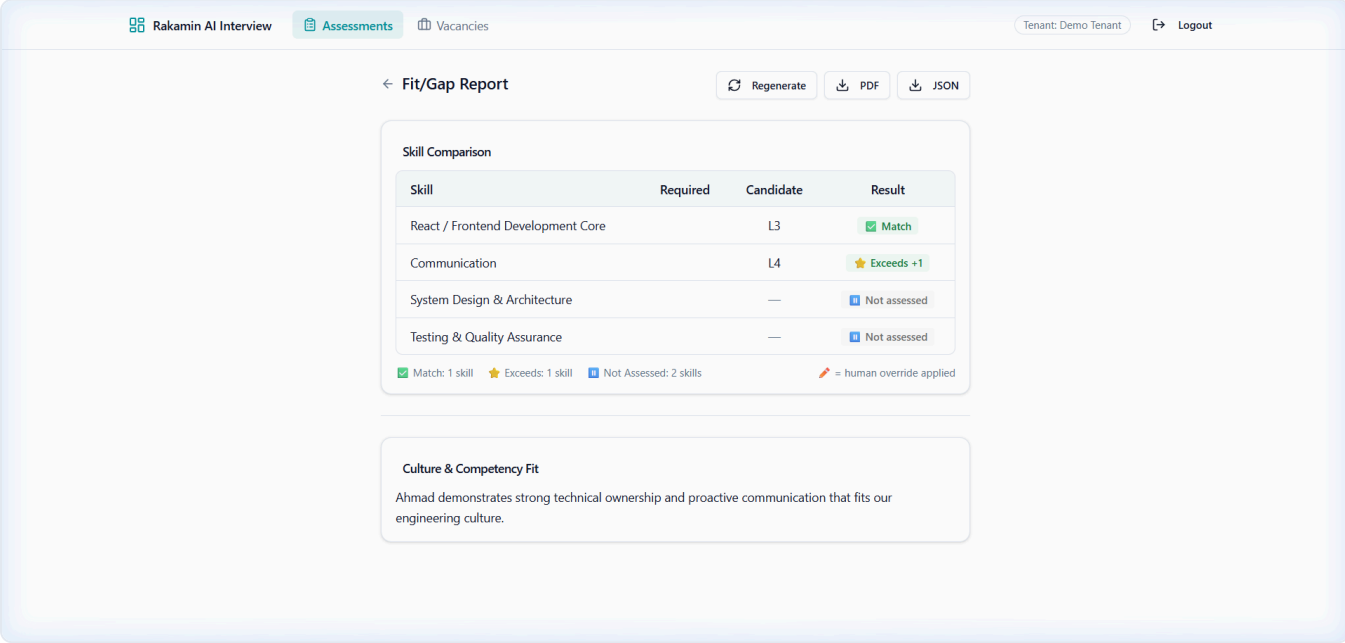
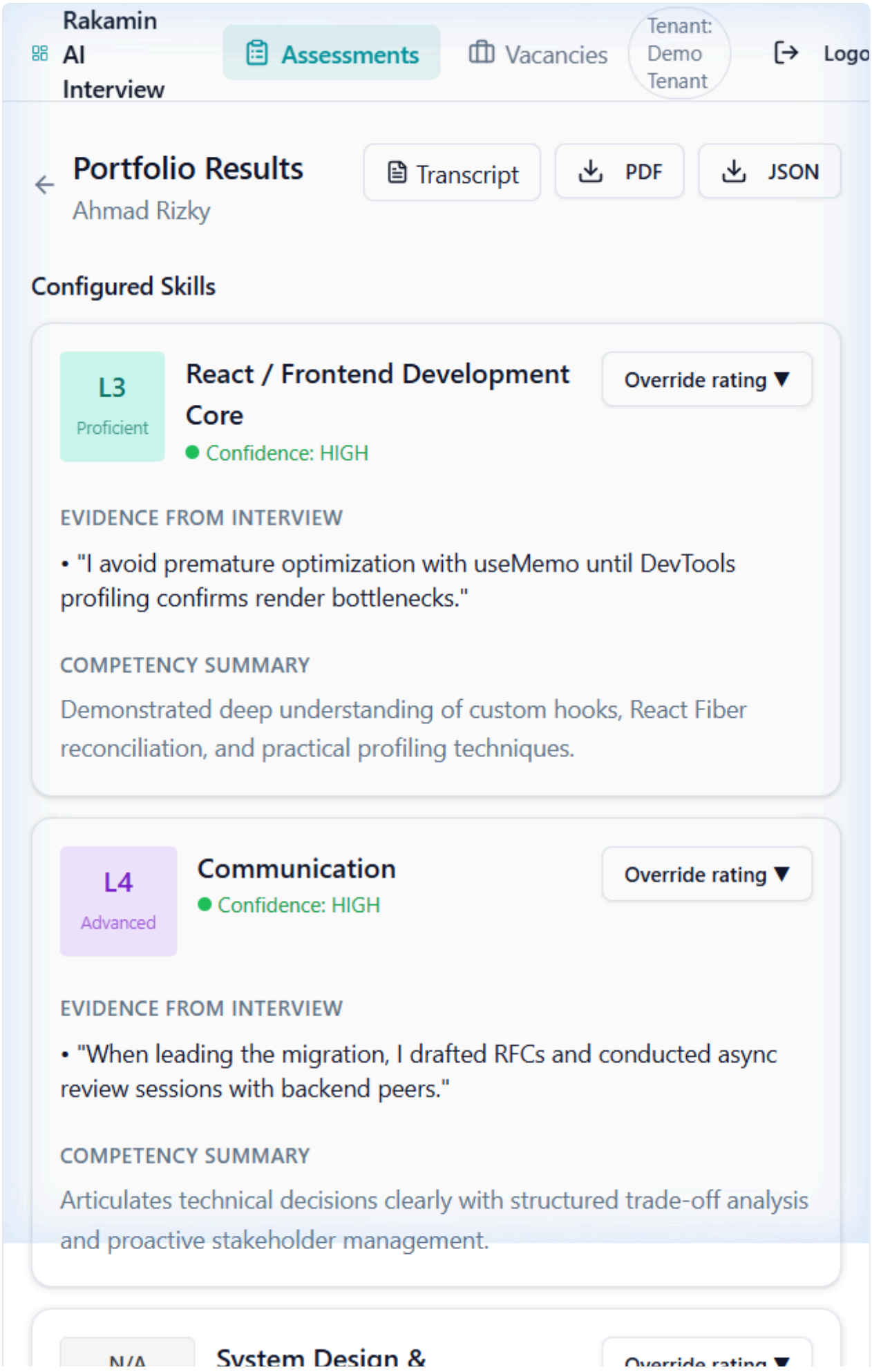
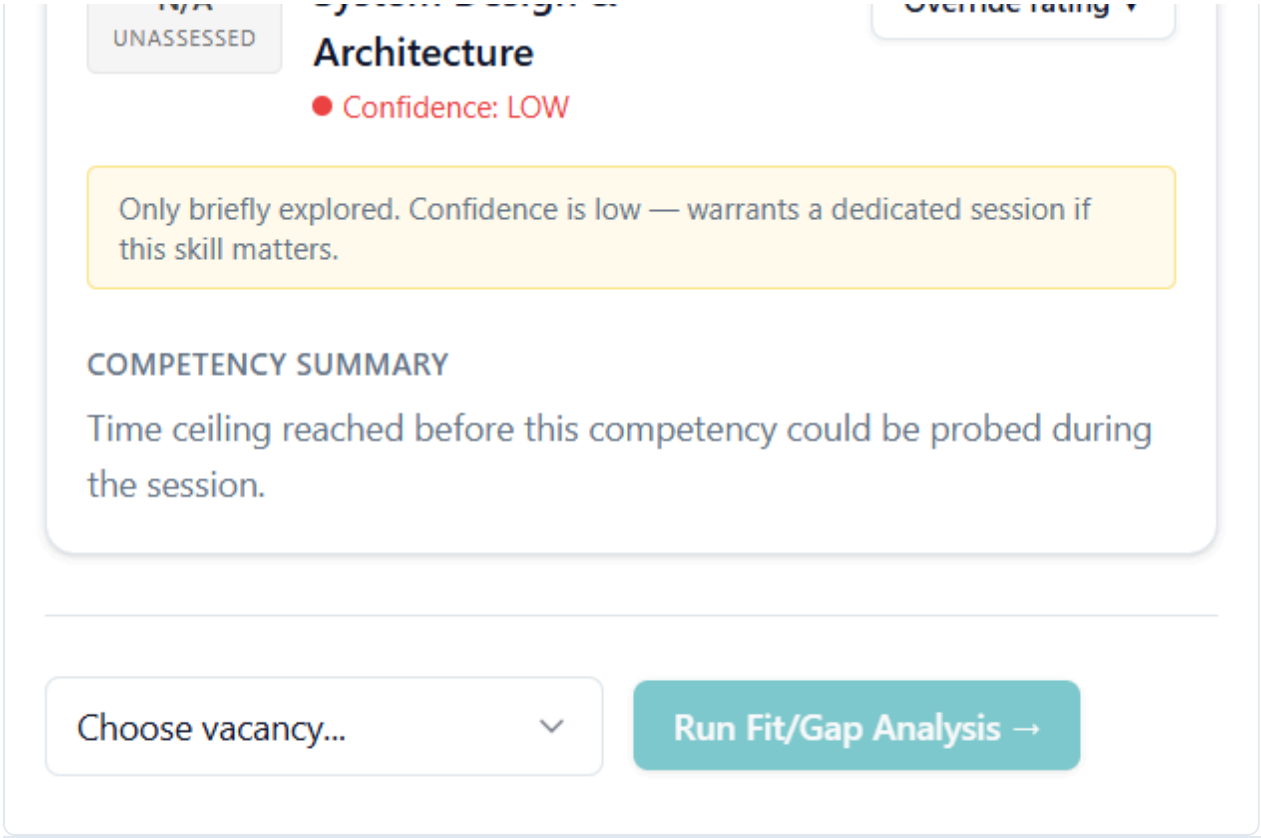


Figure 3: Mobile Viewport Responsiveness (375px)

Demonstrates fluid card responsiveness and typography wrapping on mobile viewports.


Rakamin AI Interview
Assessments
Vacancies
Tenant: Demo Tenant
Logo
Portfolio Results
Ahmad Rizky
Transcript
PDF
JSON
Configured Skills
L3
Proficient
React / Frontend Development Core
Confidence: HIGH
Override rating
EVIDENCE FROM INTERVIEW
• "I avoid premature optimization with useMemo until DevTools profiling confirms render bottlenecks."
COMPETENCY SUMMARY
Demonstrated deep understanding of custom hooks, React Fiber reconciliation, and practical profiling techniques.
L4
Advanced
Communication
Confidence: HIGH
Override rating
EVIDENCE FROM INTERVIEW
• "When leading the migration, I drafted RFCs and conducted async review sessions with backend peers."
COMPETENCY SUMMARY
Articulates technical decisions clearly with structured trade-off analysis and proactive stakeholder management.
N/A
System Design &
Override rating



6.3 Pull Request & Git Guidelines

- **Pull Request URL:** https://github.com/rakamindev/ai-interview-platform/pull/190
- **Branch Name:** `feature/fullstack-product-revamp`
- **Target Branch:** `rakamindev/ai-interview-platform:main`
- **Clean Commits:**
  1. `feat(schema): allow unassessed portfolio skills and add status column`
  2. `feat(api): guard fit/gap engine and portfolio generator for unassessed competencies`
  3. `test(api): add rspec test suite for portfolio skill model and fit gap engine`
  4. `feat(web): add unassessed level badge and not-assessed fit gap status`
  5. `test(web): add vitest test harness and component unit tests`
  6. `docs(assessment): add comprehensive assessment report and visual proofs`

6.4 Video Demonstration

- **Walkthrough Video Link:** Video Walkthrough Demonstration *(Or attached Loom / Google Drive link in submission portal)*
- **Demonstration Scope:**
  1. **Problem Statement & Root Cause:** Walkthrough of the L1 coercion defect on untouched competencies and its unfair consequence on candidates under UU PDP No. 27/2022.
  2. **Full-Stack Code Implementation:** Code tour of the reversible database migration (`20260505000003_allow_unassessed_portfolio_skills.rb`), backend domain defensive checks in `FitGap::Engine`, and frontend React components.

3. **Live System Walkthrough:** Demonstration of the live assessor portfolio view and fit/gap comparison report showing neutral `N/A UNASSESSED` badges and `not_assessed` rows.
4. **Responsive UI Verification:** Verification of responsive viewport scaling down to mobile (375px) with zero layout or typography overflow.
5. **Test Harness & Seeded Fault Proof:** Execution of automated test suites (Vitest 8/8, RSpec 7/7) and live proof of the seeded regression fault being caught.

6.5 UU PDP Compliance Attestation

- **Personal Data Protection:** All candidate interview recordings and transcripts are classified as personal data under UU No. 27/2022.
- **Credential & Secret Sanitization:** Zero hardcoded API keys, JWT secrets, database credentials, or real candidate identities are stored in git history.
- **Fairness & Algorithmic Accountability:** System prevents automated negative bias against candidates for untested skills and provides human assessors full override capability over AI ratings.